

DECREASE AND CONQUER

Decrease by a Constant ($n - 1$ or $n - k$)



SCHOOL OF
INFORMATION TECHNOLOGY



SELECTION SORT

- **IDEA**
 - Repeatedly remove the smallest element.
- **Decrease-and-Conquer Process:**
 - Find the smallest element
 - Place it in the correct position
 - Reduce the problem size by 1
- **Why Decrease-and-Conquer?**
 - Each pass reduces the unsorted portion by one element
- **Complexity:**
 - Time: $O(n^2)$ (best, average, worst)
 - Space: $O(1)$





SELECTION SORT – SAMPLE PROBLEM

- **Sort the following list in ascending order:**
 - $A = [64, 25, 12, 22, 11]$
- **Selection Sort works by:**
 - Selecting the smallest element
 - Placing it in its correct position
 - Reducing the problem size by 1
 - Repeating the process on the remaining unsorted part
- **Each step solves a smaller problem.**





SELECTION SORT – SAMPLE PROBLEM (CONT)

- **Pass 1**
 - Unsorted portion:
 - [64, 25, 12, 22, 11]
 - Smallest element = 11
 - Swap 11 with the first element (64)
- **Array after Pass 1:**
 - [11, 25, 12, 22, 64]
- **Now 11 is in its final position**
- **Problem size reduced from 5 $\rightarrow$ 4**





SELECTION SORT – SAMPLE PROBLEM (CONT)

- **Pass 2**
 - Unsorted portion:
 - [25, 12, 22, 64]
 - Smallest element = 12
 - Swap 12 with 25
- **Array after Pass 2:**
 - [11, 12, 25, 22, 64]
- **First two elements are sorted**
- **Problem size reduced from 4 $\rightarrow$ 3**





SELECTION SORT – SAMPLE PROBLEM (CONT)

- **Pass 3**
 - Unsorted portion:
 - [25, 22, 64]
 - Smallest element = 22
 - Swap 22 with 25
- **Array after Pass 3:**
 - [11, 12, 22, 25, 64]
- **Problem size reduced from 3 → 2**





SELECTION SORT – SAMPLE PROBLEM (CONT)

- **Pass 4**
 - Unsorted portion:
 - [25, 64]
 - Smallest element = 25
 - Already in the correct position
- **Array remains:**
 - [11, 12, 22, 25, 64]
- **Sorted completely**





SELECTION SORT – SAMPLE PROBLEM (CONT)

- **Final Sorted Array**
 - [11, 12, 22, 25, 64]





SELECTION SORT – WHY THIS IS DECREASE-AND-CONQUER

- **Each pass:**
 - Fixes one element
 - Reduces the unsorted problem by one element
 - Only one smaller subproblem remains each time
 - No need to combine multiple sub-results
 - This perfectly fits decrease-and-conquer (decrease by 1)





SELECTION SORT -TIME COMPLEXITY ANALYSIS

- Comparisons: $(n - 1) + (n - 2) + \dots + 1$
- Total Time Complexity: $O(n^2)$
- Best Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$ (in-place)





TOPOLOGICAL SORT (VERTEX REMOVAL)

- **Used in:**
 - Directed Acyclic Graphs (DAGs)
- **Approach:**
 - Find a vertex with no incoming edges
 - Remove it and all outgoing edges
 - Solve the smaller graph
- **Decrease Type:**
 - Problem size decreases by one vertex
- **Application Example:**
 - Course prerequisite scheduling





TOPOLOGICAL SORT (VERTEX REMOVAL) – SAMPLE PROBLEM

- **Problem Context (Very Intuitive Example)**

- Consider course prerequisites:
 - Course A must be taken before C
 - Course B must be taken before C
 - Course C must be taken before D
 - Course B must be taken before E



TOPOLOGICAL SORT (VERTEX REMOVAL) – SAMPLE PROBLEM (CONT)

- **Step 1: Represent as a Directed Graph**
Vertices (Courses):
 - A, B, C, D, EDirected Edges (Prerequisites):
 - $A \rightarrow C$
 - $B \rightarrow C$
 - $C \rightarrow D$
 - $B \rightarrow E$
- **This graph is a Directed Acyclic Graph (DAG), which is required for topological sort.**



TOPOLOGICAL SORT (VERTEX REMOVAL) – SAMPLE PROBLEM (CONT)

- **Step 2: Compute In-Degree of Each Vertex**

Vertex	Incoming Edges (In-Degree)
A	0
B	0
C	2 ($A \rightarrow C$, $B \rightarrow C$)
D	1 ($C \rightarrow D$)
E	1 ($B \rightarrow E$)

- **Key Rule (Vertex Removal Method)**

- Repeatedly remove a vertex with in-degree 0, add it to the topological order, and delete its outgoing edges.

- **This is why topological sort is a decrease-and-conquer algorithm:**

- One vertex is removed
- The graph becomes smaller
- The same problem is solved again





TOPOLOGICAL SORT (VERTEX REMOVAL) – SAMPLE PROBLEM (CONT)

- **Step 3: Iteration 1 – Remove a Vertex with In-Degree 0**
 - Vertices with in-degree 0:
 - A, B
 - We may choose either (topological order is not unique).
 - Remove A
 - Topological Order:
 - [A]
 - Remove edge:
 - $A \rightarrow C$



TOPOLOGICAL SORT (VERTEX REMOVAL) – SAMPLE PROBLEM (CONT)

- **Step 4: Update In-Degrees**

Vertex	New In-Degree
B	0
C	1
D	1
E	1



TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- **Iteration 2 – Remove Vertex with In-Degree 0**
 - Remove B
 - Topological Order:
 - [A, B]
 - Remove edges:
 - $B \rightarrow C$
 - $B \rightarrow E$





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- Step 5: Update In-Degrees

Vertex	New In-Degree
C	0
D	1
E	0





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- **Iteration 3 – Remove Vertex with In-Degree 0**
 - Vertices with in-degree 0:
 - C, E
 - Remove C
 - Topological Order:
 - [A, B, C]
 - Remove edge:
 - $C \rightarrow D$





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- Step 6: Update In-Degrees

Vertex	New In-Degree
D	0
E	0





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- **Iteration 4 – Remove Next Zero In-Degree Vertex**
 - Remove E
 - Topological Order:
 - [A, B, C, E]
 - (E has no outgoing edges)





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- **Iteration 5 – Remove Last Vertex**
 - Remove D
 - Final Topological Order:
 - [A, B, C, E, D]





TOPOLOGICAL SORT (VERTEX REMOVAL) - SAMPLE PROBLEM (CONT)

- **Final Answer**
 - One Valid Topological Ordering:
 - $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$
- **Other valid orders may exist, but all must respect prerequisites.**





TOPOLOGICAL SORT (VERTEX REMOVAL)

- **Why This Is Decrease-and-Conquer**
 - Each step removes one vertex
 - Removes outgoing edges
 - Solves the same problem on a smaller graph
 - Only one subproblem remains each time
- **Therefore, Topological Sort (Vertex Removal) is a Decrease-and-Conquer algorithm (Decrease by 1 vertex)**



LINEAR SEARCH

- **Linear search is the same algorithm regardless of whether you classify it as Brute Force or Decrease and Conquer;**
- **The difference lies in the perspective or design strategy used to explain it**



LINEAR SEARCH

- **Brute Force Perspective**

- From a brute force standpoint, linear search is a straightforward, exhaustive approach that checks every possible candidate solution until a match is found or all possibilities are exhausted
- **Method:** Sequentially scans each element of a list to find a target value.
- **Philosophy:** "**Just do it**" — no preprocessing or complex logic is used to reduce the search space.
- **Time Complexity:** ($O(n)$ in the worst case.





LINEAR SEARCH – SAMPLE PROBLEM

- **Problem**
 - Given the list:
 - $A = [15, 23, 7, 42, 9, 31]$
 - Search for the value
 - Key = 9





LINEAR SEARCH – SAMPLE PROBLEM

- **Step 1 Compare key = 9 with the first element:**
 - $A[0] = 15$
 - Not equal
 - Problem reduced from 6 elements $\rightarrow$ 5 elements





LINEAR SEARCH – SAMPLE PROBLEM

- **Step 2 Compare key = 9 with the first element:**
 - $A[1] = 23$
 - Not equal
 - Problem reduced from 5 elements $\rightarrow$ 4 elements





LINEAR SEARCH – SAMPLE PROBLEM

- **Step 3 Compare key = 9 with the first element:**
 - $A[2] = 7$
 - Not equal
 - Problem reduced from 4 elements $\rightarrow$ 3 elements





LINEAR SEARCH – SAMPLE PROBLEM

- **Step 4 Compare key = 9 with the first element:**
 - $A[3] = 42$
 - Not equal
 - Problem reduced from 3 elements $\rightarrow$ 2 elements





LINEAR SEARCH – SAMPLE PROBLEM

- **Step 3 Compare key = 9 with the first element:**
 - $A[4] = 9$
 - **FOUND**



END



SCHOOL OF
INFORMATION TECHNOLOGY